

COMPUTER NETWORKS

Units 1 and 2

Conceptual notes, diagrams, formulas, and solved previous-year questions

Designed for university examination writing

The explanations are intentionally written in complete sentences so that you can reproduce them in a 5-mark or 10-mark answer. Learn the definition, draw the diagram, write the working, then close with the comparison or conclusion.

Coverage

Unit 1: Introduction to the Internet, network edge and core, access networks, physical media, switching, ISPs, delay/loss/throughput, security, protocol layering, OSI and TCP/IP.

Unit 2: Application architectures, processes and sockets, transport services, HTTP, cookies and caching, e-mail, DNS, P2P, video streaming, DASH, CDNs, and socket programming.

Source note

The supplied PPTs are based on Computer Networking: A Top-Down Approach, 9th edition, by Jim Kurose and Keith Ross. These notes explain the same syllabus in simpler exam language and add worked reasoning where the slides are terse.

How to write a 10-mark answer

Use the following pattern whenever the question asks explain, discuss, compare, or describe:

- Start with a one or two sentence definition.
- Draw a labelled diagram. A simple correct diagram earns clarity even when the wording is long.
- Explain the working in numbered steps, using the vocabulary of the protocol or layer.
- Add advantages, limitations, formulas, or a comparison table when the question asks for them.
- End with one short conclusion that connects the mechanism to its purpose.

High-value exam habit

Do not write only keywords such as reliable, fast, or scalable. Explain what mechanism produces that property. For example, TCP is reliable because it uses sequence numbers, acknowledgements, timers, retransmissions, and checksum-based error detection.

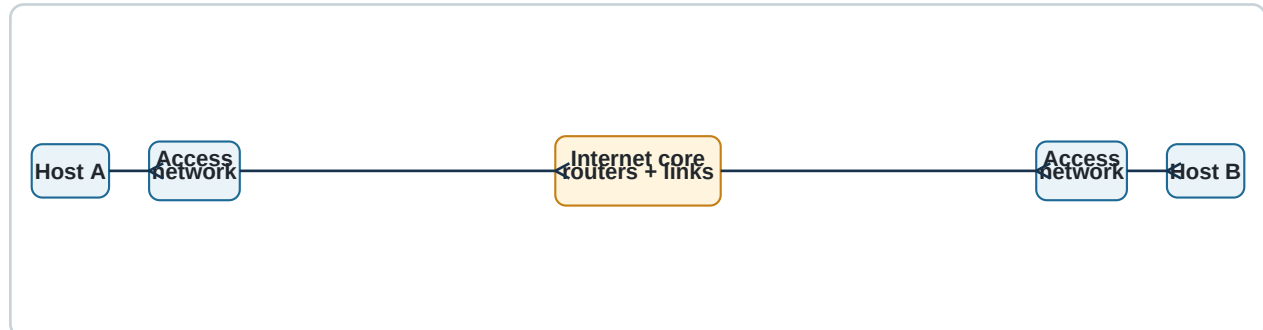
Course map

Unit	Core areas	Typical question verbs
1	Internet basics, edge/core, switching, delay, performance, layers	Define, compare, calculate, trace, explain
2	HTTP, SMTP/IMAP, DNS, P2P, streaming/CDN, sockets	Describe, differentiate, trace, justify, illustrate

UNIT 1 | INTRODUCTION TO COMPUTER NETWORKS

1.1 The Internet: two useful views

The Internet is a global network of interconnected networks. It connects billions of end systems such as laptops, phones, servers, sensors, and smart devices using communication links and packet switches. Routers and switches forward packets from a source to a destination.



A simplified view of hosts, access networks, and the Internet core

Nuts-and-bolts view. The Internet contains end systems, communication links, packet switches, routers, switches, and networks operated by different organizations. The links may use copper, fibre, radio, or satellite.

Services view. The Internet provides a programming interface to distributed applications. Web browsing, e-mail, video streaming, games, social media, and video calls use transport services supplied by the network. An application sends data to a socket and receives data from a socket.

1.2 What is a protocol?

A protocol is a set of rules that defines the format and order of messages exchanged between communicating entities and the actions taken when a message is sent or received. Human conversation also follows protocols, but computer-network protocols must be precise enough for machines to implement.

- Message syntax: the structure and fields of a message.
- Message semantics: the meaning of each field.
- Timing and actions: when a message is sent and what the receiver does next.
- Examples: HTTP for the Web, SMTP for mail transfer, DNS for name resolution, TCP and IP for transport and network communication.

Exam-ready definition

A network protocol specifies the format, order, and actions associated with messages exchanged by network entities. It enables independent devices and software implementations to communicate correctly.

1.3 Network edge: hosts, clients, and servers

The network edge contains end systems, also called hosts. A host runs application processes and is the original source or final destination of application data. A client process usually initiates a request. A server process usually waits for requests and provides a service.

Term	Meaning	Example
Host / end system	A device at the edge of the network that runs applications and sends or receives data.	Laptop, smartphone, web server, sensor
Client	A process that initiates communication to obtain a service.	Browser requesting a Web page
Server	A process that waits for requests and sends a response or service.	Apache server returning an HTML file

A Web server is an end system when it is a physical or virtual host connected to the Internet. The word server can also mean the application process running on that host.

1.4 Access networks and physical media

An access network connects an end system to its first router, often called the edge router. The access link determines how the user enters the Internet and often becomes the bottleneck.

Access network	Working and exam points
Cable / HFC	Uses a hybrid fibre-coax distribution network. Many homes share the cable segment to a cable headend. Frequency division multiplexing separates data and television channels. It is usually asymmetric, with higher downstream than upstream capacity.
DSL	Uses the existing telephone line. A DSL modem and DSLAM separate voice and data and connect the subscriber to the central office. The access line is normally dedicated to the subscriber.
Home network	A modem/router often combines routing, firewall, NAT, wired Ethernet, and Wi-Fi access-point functions.
Wi-Fi / cellular	A wireless access point or cellular base station connects mobile devices to a router or carrier network. Wi-Fi covers a building-scale area; 4G/5G covers a much larger geographic area.
Enterprise / campus	Mixes Ethernet switches, Wi-Fi access points, routers, and servers. Data centers use high-bandwidth links to connect many servers.

Physical media

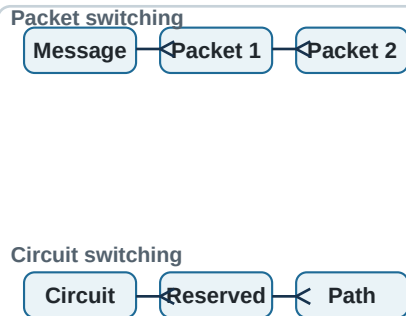
Guided media carry signals through a solid medium. Unguided media carry radio signals through the air. The choice affects rate, distance, interference, cost, and mobility.

Medium	Main characteristics
Twisted pair	Two insulated copper wires. Common in Ethernet. Category 5/6 cables support increasingly higher rates over short distances.
Coaxial cable	Two concentric copper conductors. Supports broadband transmission and multiple frequency channels.
Fibre optic	Glass fibre carries light pulses. Very high rate, long distance, low error rate, and resistance to electromagnetic noise.
Radio	No physical wire. Supports Wi-Fi, cellular, Bluetooth, microwave, and satellite. Reflection, obstruction, interference, and propagation conditions affect performance.

UNIT 1 | NETWORK CORE AND SWITCHING

1.5 The network core

The network core is a mesh of interconnected routers. End systems break application messages into packets, and routers forward packets over a path to the destination.



Packet switching breaks data into packets; circuit switching reserves a path or capacity

Forwarding is the local action of moving an arriving packet from an input link to the correct output link using a forwarding table. **Routing** is the network-wide process of determining the path from source to destination using routing algorithms.

Packet switching

In packet switching, a message is divided into packets. Each packet carries a destination address in its header and may share links with packets from other users. A router normally uses store-and-forward: it receives the complete packet before transmitting it on the next link.

- Transmission delay for an L-bit packet on an R-bit/s link is L/R seconds.
- Queueing occurs when packets arrive at an output link faster than the link can transmit them.
- A finite buffer can overflow. A packet arriving when the buffer is full is dropped, producing packet loss.
- Packet switching is efficient for bursty traffic because link capacity is shared on demand.
- Congestion can produce variable delay, loss, and retransmissions.

Circuit switching

In circuit switching, the network reserves end-to-end resources for a call before data transfer. The reservation may be a frequency band in FDM or periodic time slots in TDM. Once established, the circuit provides predictable service, but reserved capacity is idle when the user has nothing to send.

Point	Packet switching	Circuit switching
Resource allocation	On demand; shared	Reserved for a call
Setup	Usually no dedicated call setup	Call setup is required
Performance	Delay and loss can vary with congestion	More predictable after setup
Efficiency	Good for bursty data	Can waste capacity during silence
Typical use	Internet data traffic	Traditional telephone networks

FDM and TDM

In Frequency Division Multiplexing, the link spectrum is divided into frequency bands and each call receives a band. In Time Division Multiplexing, time is divided into repeating slots and each call receives one or more slots. TDM allows each user to transmit at the full link rate during the assigned slot, while FDM limits a user to the rate of the allocated band.

Three phases of circuit switching

- Setup phase: reserve resources along the path and establish the circuit.

- Data-transfer phase: send data using the reserved circuit.
- Teardown phase: release the resources after communication ends.

Circuit-switching advantage

A circuit-switched network offers guaranteed or predictable performance after setup because the resources are dedicated. Its main disadvantages are setup delay, wasted capacity for idle users, and limited flexibility during bursty traffic.

1.6 Internet structure: a network of networks

Access ISPs connect homes, enterprises, and mobile users. Regional ISPs connect access ISPs. Tier-1 or global transit ISPs provide wide-area connectivity. Internet exchange points allow networks to peer directly. Large content providers such as video and cloud companies may operate private networks and place servers near users.

Component	Role
Access ISP	Provides the last-mile connection to homes, campuses, businesses, or mobile users.
Regional ISP	Aggregates access networks and connects them to larger providers or peers.
Tier-1 ISP	Large national or international backbone with extensive interconnection.
IXP / peering	A meeting point where independent networks exchange traffic directly.
Content provider network / CDN	Private infrastructure that places content and services close to end users.

For a small business, a Tier-2 or regional ISP is usually the sensible choice: it can provide reliable business access and support at lower cost than a Tier-1 backbone. A Tier-1 connection is justified only when the organization itself needs large-scale backbone reach or transit service.

UNIT 1 | DELAY, LOSS, AND THROUGHPUT

1.7 Four components of nodal delay

When a packet travels through a router, its nodal delay is the sum of processing, queueing, transmission, and propagation delay.



Four delay components at a link

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

Component	Meaning	Formula / behaviour
Processing delay	Time to examine the header, check errors, and select the output link.	Usually small and approximately constant for a given device.
Queueing delay	Time waiting in the output buffer before transmission begins.	Variable; grows with traffic intensity and congestion.
Transmission delay	Time to push all L bits into the link.	$d_{\text{trans}} = L / R$
Propagation delay	Time for a bit to travel through the physical medium.	$d_{\text{prop}} = d / s$

Transmission and propagation are different. A high-bandwidth link reduces transmission delay, while a shorter physical distance or faster medium reduces propagation delay. Increasing bandwidth does not automatically reduce propagation delay.

Queueing and loss

Let λ be the average packet arrival rate, L the average packet length, and R the link rate. The traffic intensity is $\lambda L / R$. When this value is close to zero, queueing is small. As it approaches 1, queueing becomes large. If the input rate stays above the service rate, the buffer can fill and packets are dropped.

Packet loss can be handled by retransmission from the previous router or source, or it may be left to the application. TCP uses retransmission and congestion control; UDP itself does not provide them.

Throughput

Throughput is the rate at which bits are delivered from a sender to a receiver. The end-to-end throughput is limited by the bottleneck link, the slowest effective rate on the path.

$$\text{Throughput} = \min(\text{sender rate}, \text{receiver access rate}, \text{bottleneck capacity})$$

- Instantaneous throughput is measured at a particular time.
- Average throughput is measured over a longer interval.
- If a server can send at R_s and the client link can receive at R_c , the ideal throughput is $\min(R_s, R_c)$.
- If ten connections share a backbone link fairly, each may receive approximately $R/10$, subject to other bottlenecks.

Worked numerical example

A 1500-byte packet (12000 bits) travels from Host A to Host B over one link. The rate is 10 Mbps, distance is 1000 km, propagation speed is 2×10^8 m/s, queueing delay is 2 ms, and processing delay is 1 ms.

$$\text{Transmission} = 12000 / 10,000,000 = 1.2 \text{ ms}$$

$$\text{Propagation} = 1,000,000 / (2 \times 10^8) = 5 \text{ ms}$$

$$\text{End-to-end delay} = 1.2 + 5 + 2 + 1 = 9.2 \text{ ms}$$

In an exam, show the unit conversion from kilometres to metres and from bits per second to the numeric link rate.

1.8 Traceroute idea

Traceroute estimates the delay to each router on a path. It sends probes with increasing time-to-live values. The router whose TTL expires returns a response, and the sender measures the round-trip time. A star can mean the probe was lost or the router did not reply.

UNIT 1 | SECURITY, LAYERS, AND ENCAPSULATION

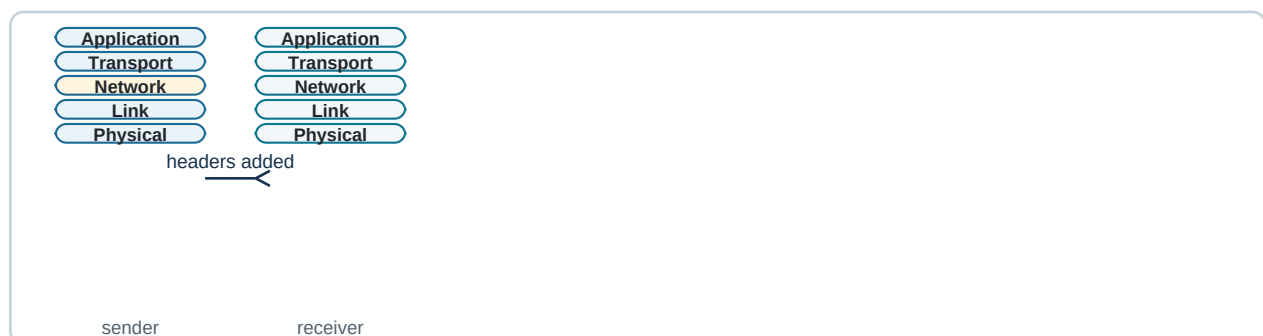
1.9 Basic network security threats

Threat	Meaning	Defence idea
Packet sniffing	A device on a shared medium reads packets that pass by, possibly exposing passwords or data.	Encryption, secure protocols, and switched or protected links.
IP spoofing	An attacker injects a packet with a false source address.	Authentication, filtering, and secure protocol checks.
Denial of Service	Attackers overwhelm a server, link, or service with bogus traffic so legitimate users cannot use it.	Firewalls, filtering, rate limiting, monitoring, and distributed defence.

- Authentication proves who a communicating party is.
- Confidentiality prevents outsiders from reading data, usually through encryption.
- Integrity detects unauthorised modification, for example with message authentication or digital signatures.
- Access control restricts senders, receivers, applications, or ports. A firewall is a specialised middlebox that filters traffic.

1.10 Why protocol layering?

Networking is a complex system containing applications, protocols, software, hosts, routers, links, and media. Layering divides the system into modules. Each layer offers a service to the layer above and uses a service from the layer below.



A simplified TCP/IP stack at two end systems

Advantages of layering

- Modularity: one layer can be designed and tested separately.
- Maintenance: changes inside one layer can remain hidden from other layers.
- Standardisation: different vendors can implement the same layer interface.
- Troubleshooting: the location of a problem can be narrowed to a layer.
- Trade-off: strict layering can add overhead or duplicate functions.

OSI reference model

OSI layer	Main responsibility	Examples / note
Application	Network applications and application protocols.	HTTP, SMTP, DNS
Presentation	Data representation, translation, compression, encryption.	Often implemented inside applications in the Internet stack.
Session	Dialog control, synchronisation, checkpoints, recovery.	Often implemented inside applications.
Transport	Process-to-process delivery, reliability, flow and congestion control.	TCP, UDP
Network	Host-to-host delivery and routing of packets.	IP, routing protocols
Data link	Frame delivery between neighbouring network elements.	Ethernet, Wi-Fi, PPP
Physical	Transmission of raw bits over the medium.	Copper, fibre, radio

TCP/IP Internet stack

The practical Internet stack normally uses application, transport, network, link, and physical layers. The OSI presentation and session functions are usually implemented inside the application layer rather than as separate Internet layers.

1.11 Encapsulation

At the sender, each lower layer adds its own header to the data from the layer above. At the receiver, the headers are removed in reverse order. The names are message at the application layer, segment at transport, datagram at network, and frame at link.

Encapsulation at sender



Encapsulation adds a header at each lower layer

Example: if an application message is M, the transport layer forms [H_t | M], the network layer forms [H_n | H_t | M], and the link layer forms [H_l | H_n | H_t | M]. Routers normally remove and add link-layer frames at each hop, while the transport and application data remain end-to-end.

UNIT 1 | HISTORY AND QUICK REVISION

1.12 Short Internet history

Period	Milestone and importance
1961-1972	Queueing theory and early packet-switching ideas; ARPANET conceived and first nodes operated; early host-to-host protocols and e-mail.
1972-1980	Internetworking principles, Ethernet, and the growth of independent networks.
1980-1990	TCP/IP deployment, DNS, SMTP, FTP, congestion control, and new national networks.
1990s-2000s	ARPANET retired; commercial Web, HTML, HTTP, browsers, search, messaging, and P2P applications expanded.
2005-present	Broadband, Wi-Fi/4G/5G, cloud services, software-defined networking, content-provider networks, and large-scale mobile use.

Unit 1 formula sheet

Quantity	Formula
Transmission delay	$d_{trans} = L / R$
Propagation delay	$d_{prop} = d / s$
Nodal delay	$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$
Traffic intensity	La / R
End-to-end throughput	approximately the minimum rate on the path
Full mesh pair circuits	$n(n - 1) / 2$

Unit 1 last-minute checklist

Know host versus end system; packet versus circuit switching; FDM versus TDM; forwarding versus routing; four delay components; throughput bottleneck; security threats; OSI/TCP-IP mapping; and encapsulation.

UNIT 2 | APPLICATION LAYER

2.1 Principles of network applications

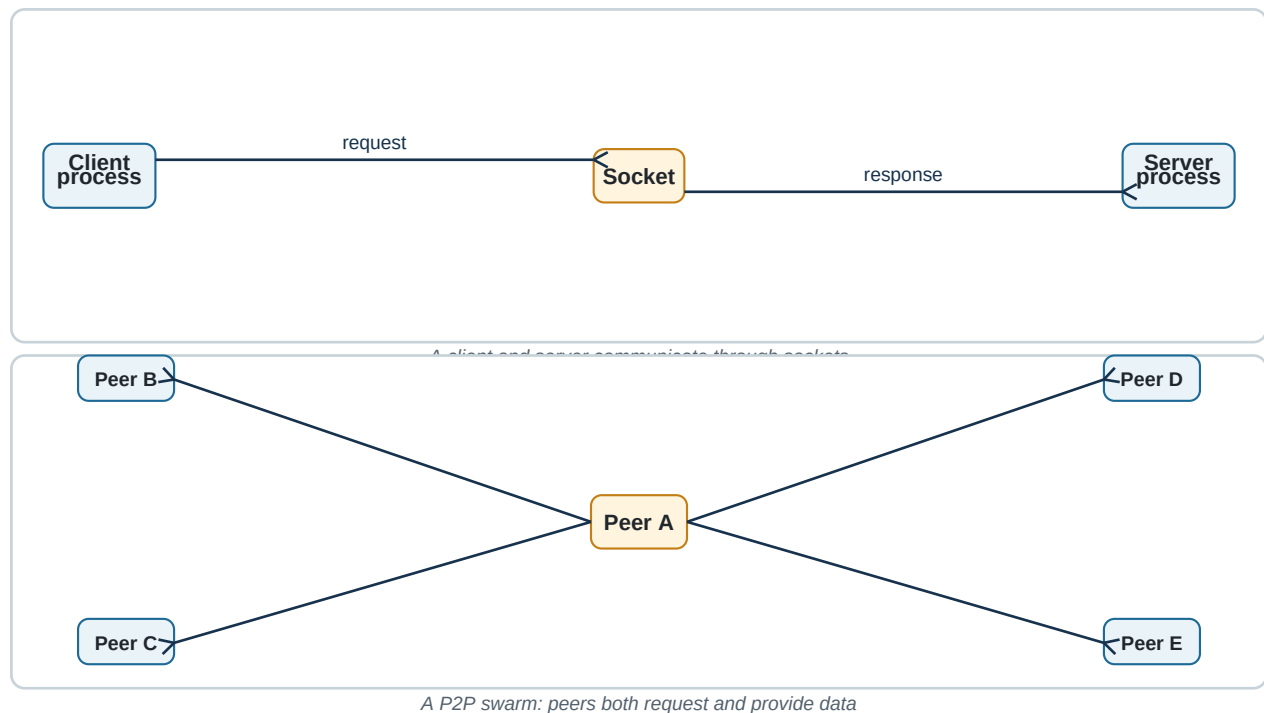
A network application is a program that runs on end systems and communicates with another program over the network. Developers normally do not write application code inside routers. This separation allows rapid application development at the network edge.

Client-server architecture

A server is often always on, has a stable or well-known address, and may run in a data center. Clients contact the server and may be intermittently connected with changing addresses. HTTP, FTP, and traditional mail access are examples.

Peer-to-peer architecture

In P2P, end systems communicate directly. A peer can request data from other peers and upload data to them. There is no requirement for one always-on central server. New peers add both demand and upload capacity, giving P2P self-scalability, but peer discovery, security, availability, and management are more difficult.



Processes and sockets

A process is a running program. Processes on different hosts exchange messages. A socket is the interface between an application process and the transport layer. It is like a door: the application controls one side, while the operating system and transport protocol deliver data on the other side.

An IP address identifies a host, not a unique process. A process is identified by the combination of IP address and port number. Common examples are HTTP port 80, HTTPS port 443, and SMTP port 25.

An application-layer protocol defines message types, syntax, semantics, and rules for sending and responding. Open protocols are published in RFCs. Proprietary protocols are controlled by a company or product.

UNIT 2 | TRANSPORT SERVICES FOR APPLICATIONS

2.2 What an application needs from transport

Requirement	Meaning	Examples
Reliability / data integrity	Data should arrive correctly and completely.	File transfer, e-mail, Web transactions
Timing	Delay should be small enough for the application to remain effective.	Interactive games, voice, live video
Throughput	The application needs a minimum or useful data rate.	Video streaming, multimedia
Security	The application may need confidentiality, authentication, and integrity.	Payments, login, private messages

TCP and UDP

Feature	TCP	UDP
Data transfer	Reliable byte stream, in order	Unreliable datagrams; loss or reordering may occur
Connection	Connection-oriented; setup required	Connectionless; no handshake before sending
Control	Flow control and congestion control	No built-in flow or congestion control
Guarantees	No timing or minimum-throughput guarantee	No reliability, timing, throughput, or security guarantee
Use	HTTP, SMTP, IMAP, file transfer	DNS queries, streaming or real-time apps when chosen, games, custom apps

TCP is reliable because it uses checksum, sequence numbers, acknowledgements, timers, retransmissions, and ordered delivery. UDP is useful when an application wants low overhead, message boundaries, fast startup, or direct control of loss and timing. UDP itself does not make an application secure; TLS or another security layer is needed.

TLS

A normal TCP or UDP socket carries cleartext unless the application adds security. TLS provides encrypted communication, data integrity, and endpoint authentication. HTTPS is HTTP carried over a secure TLS connection, commonly over TCP; HTTP/3 uses QUIC, which integrates security and transport functions over UDP.

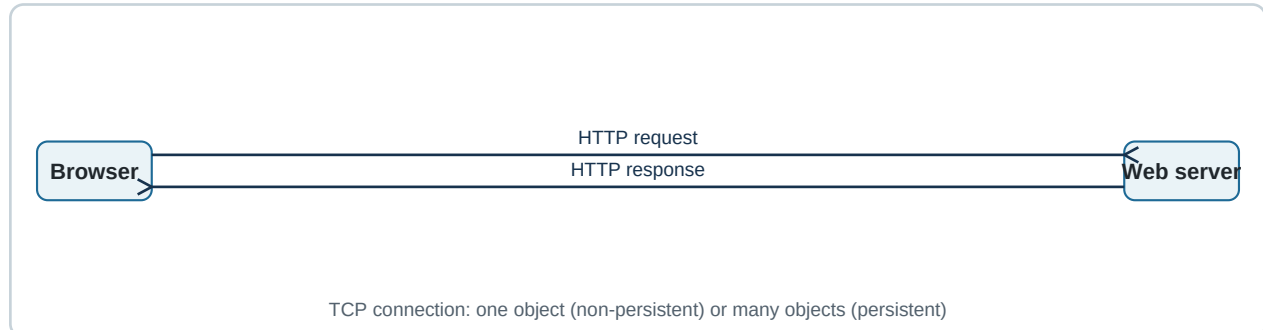
2.3 Why HTTP, SMTP, and IMAP use TCP

These protocols transfer Web objects or e-mail where loss, duplication, or reordering would corrupt the application data. TCP supplies reliable in-order delivery, retransmission, flow control, and congestion control. The application therefore does not need to implement those functions itself.

UNIT 2 | WEB AND HTTP

2.4 Web pages, URLs, and HTTP

A Web page consists of a base HTML file and referenced objects such as images, style sheets, scripts, and video. Each object has a URL containing a host name and a path. HTTP is the application-layer protocol used by a browser and Web server to request and transfer these objects.



HTTP request and response over a TCP connection

HTTP is stateless: the server does not automatically retain information about earlier requests. Statelessness simplifies server recovery and scaling, but applications often add state through cookies, databases, or sessions.

Non-persistent and persistent HTTP

Property	Non-persistent HTTP	Persistent HTTP
TCP use	A new TCP connection for each object	One connection can carry multiple objects
Per-object delay	Approximately 2 RTT + object transmission time	Later objects avoid repeated connection setup
Cost	More handshakes and operating-system overhead	Lower overhead and better performance
Typical exam point	A page with base HTML plus 10 images may need 11 connections if fetched sequentially.	HTTP/1.1 keep-alive allows reuse; browsers may still use concurrency.

Non-persistent HTTP working

- The user enters a URL. The client resolves the host name and opens a TCP connection to the server, usually port 80 for HTTP.
- The browser sends a GET request containing the path.
- The server sends an HTTP response containing the object and closes the TCP connection.
- The browser parses the HTML. For each referenced object, it repeats the connection, request, response, and close sequence.
- For one object, response time is approximately 2 RTT plus the file transmission time: one RTT to establish TCP and one RTT for the request and first response bytes.

HTTP request format

A request contains a request line, header lines, a blank line, and an optional entity body. Example:

```
GET /index.html HTTP/1.1
Host: www.example.com
Connection: keep-alive
Accept: text/html
```

Common methods: GET retrieves an object; POST sends form data in the body; HEAD requests only the headers; PUT uploads or replaces an object at a URL.

HTTP response format and status codes

Code	Meaning
200 OK	Request succeeded; the object may follow.
301 Moved Permanently	The new location is supplied in a Location header.
304 Not Modified	The cached copy is still valid; no object body is sent.
400 Bad Request	The server cannot understand the request.
404 Not Found	The requested resource does not exist at the server.
505 HTTP Version Not Supported	The server does not support the requested HTTP version.

UNIT 2 | COOKIES, CACHES, AND HTTP EVOLUTION

2.5 Cookies and state

Cookies add state to a stateless HTTP exchange. A site sends a Set-Cookie header in a response. The browser stores the value and sends it in a Cookie header on later requests to the same site. The site can use the identifier to find session data in a backend database.

- Cookie response header: Set-Cookie: 1678
- Cookie request header: Cookie: 1678
- Browser cookie file stores the value.
- The Web site database stores the associated account, cart, or session state.

Uses include login sessions, shopping carts, authorisation, and personalised recommendations. First-party cookies come from the site the user visits. Third-party cookies come from an embedded tracker or advertisement and can correlate browsing across sites, creating privacy concerns.

2.6 Web caches / proxy servers

A Web cache is both a server to the browser and a client to the origin server. The browser sends a request to the cache. If the object is present and fresh, the cache returns it immediately. Otherwise, the cache requests the object from the origin server, stores a copy, and returns it to the browser.

- The cache is closer to the client, reducing response time.
- It reduces traffic on the institution access link and the origin path.
- It helps a smaller content provider serve many users.
- The cache must respect freshness and cache-control information.

Conditional GET

A browser can ask whether its cached copy is still current by sending If-Modified-Since. If the object has not changed, the server sends 304 Not Modified and no object body. If it has changed, the server sends 200 OK and the new object.

2.7 HTTP/2 and HTTP/3

HTTP/1.1 pipelining can suffer head-of-line blocking: a small object waits behind a large earlier response. HTTP/2 divides objects into frames and interleaves frame transmission. The server can schedule frames using priorities and may push useful objects. This reduces application-level blocking, although TCP loss recovery can still stall all streams sharing the connection.

HTTP/3 uses QUIC over UDP. QUIC provides encrypted connections, reliability, congestion control, and independent stream handling in one transport design. A normal TCP plus TLS design performs two serial

handshakes; QUIC can establish the needed state in one RTT and may support 0-RTT resumption.

UNIT 2 | E-MAIL: SMTP AND IMAP

2.8 E-mail system components

The three main components are the user agent, mail servers, and SMTP. A user agent allows a person to compose, send, and read messages. A mail server contains the user mailbox and an outgoing message queue. SMTP transfers messages between mail servers.



Mail submission, server-to-server SMTP, and recipient retrieval

SMTP uses TCP port 25 and a command-response exchange. It has a greeting or handshake, message transfer, and closure. SMTP commands and responses are ASCII and a message ends with a special CRLF.CRLF sequence.

Alice sends mail to Bob

- Alice composes a message in her user agent addressed to bob@example.edu.
- The user agent submits the message to Alice mail server, which places it in the outgoing queue.
- Alice mail server acts as an SMTP client and opens a TCP connection to Bob mail server.
- The two servers exchange HELO or EHLO, MAIL FROM, RCPT TO, DATA, and QUIT commands and responses.
- Bob mail server stores the message in Bob mailbox.
- Bob retrieves it using IMAP or a Web-mail interface over HTTP/HTTPS.

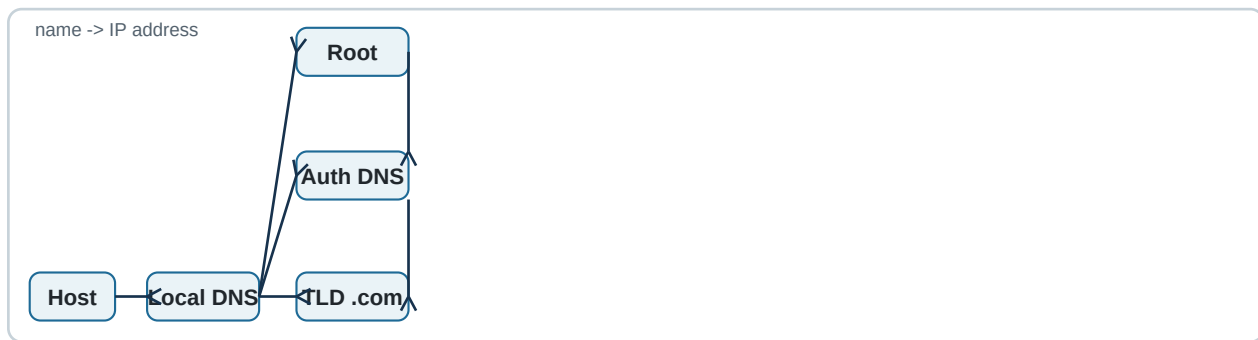
SMTP versus e-mail message format

SMTP is the protocol for transferring a message. The message itself has header fields such as To, From, and Subject, followed by a blank line and the body. MAIL FROM and RCPT TO are SMTP commands and should not be confused with the To and From header fields inside the message.

IMAP keeps messages on the mail server and supports retrieval, folders, deletion, and synchronisation. Web mail uses HTTP/HTTPS at the browser interface and uses SMTP for sending plus IMAP or a similar access service for retrieving mail.

2.9 DNS: Domain Name System

DNS maps human-readable host names to IP addresses. It is a distributed hierarchical database and an application-layer protocol. It also supports host aliasing, mail-server aliasing, and load distribution through multiple addresses.



A simplified DNS hierarchy and name-resolution path

Why DNS is distributed

- A single central server would be a single point of failure.
- A central database would receive an enormous query volume.
- A distant central server would increase response time.
- A central database would be difficult to maintain for millions of organisations.

DNS hierarchy

Root name servers direct queries to the correct top-level domain server. TLD servers handle domains such as .com, .org, .edu, and country-code domains. Authoritative name servers are responsible for the records of a particular organisation or domain. A local DNS server serves the client and may answer from its cache or contact the hierarchy.

Iterative versus recursive queries

Query type	Working	Advantage / limitation
Iterative	Each server returns the name of the next server to contact. The local resolver does the next query itself.	Distributes work and gives the resolver control, but requires several exchanges.
Recursive	The contacted server takes responsibility for resolving the complete name and returns the final answer.	Simple for the requester, but places more load on the contacted server and upper hierarchy.

Without caching, resolving `www.example.com` normally follows: client to local DNS; local DNS to root; root to .com TLD; TLD to example.com authoritative DNS; authoritative DNS returns the A record; the local DNS returns the IP to the client. With caching, an earlier answer or TLD record may remove some steps.

UNIT 2 | DNS RECORDS, P2P, VIDEO, AND CDNS

DNS caching and records

A DNS server caches a name-to-address mapping after learning it. The time-to-live (TTL) controls how long the entry remains. Caching reduces delay and query traffic, but a cached value can be temporarily stale after a host changes its address.

Record	Meaning
A	Maps a host name to an IPv4 address.
AAAA	Maps a host name to an IPv6 address.
NS	Identifies the authoritative name server for a domain.
CNAME	Maps an alias to a canonical host name.
MX	Identifies the mail server for a domain.

DNS message format

A DNS query and reply use the same general format: a header, question section, answer records, authority records, and additional records. The header has an identification number, flags, and counts for the sections. A reply may include the answer, information about the authority, and helpful additional records.

DNS security issues

DDoS attacks can target root or TLD servers. Attackers can also spoof DNS replies or poison a cache with a false mapping. DNSSEC adds authentication and integrity protection for DNS data.

2.10 P2P file sharing and BitTorrent

BitTorrent divides a file into fixed-size pieces. A group of peers sharing the same file is a swarm. A peer downloads pieces from multiple peers while uploading pieces it already has to others.

- A tracker or distributed peer-discovery mechanism helps a new peer find members of the swarm.
- A peer with no pieces asks for pieces from several neighbours. It can begin sharing a piece as soon as it obtains and verifies it.
- Rarest-first selection requests pieces that are least common among neighbours, improving piece diversity and reducing the risk that a rare piece disappears.
- Tit-for-tat or choking gives upload preference to peers that upload at a useful rate. Optimistic unchoking periodically tests another peer so a new or fast peer can be discovered.
- A peer that obtains all pieces becomes a seed and continues uploading, improving availability.
- P2P is self-scalable because every new peer adds demand but can also add upload capacity.

A client with no data should therefore contact the tracker or peer-discovery service, establish connections with several peers, request rare or available pieces, verify each piece, upload received pieces to earn service, and eventually seed the complete file.

2.11 Video streaming

Video is a sequence of images displayed at a constant rate. Compression reduces bits by exploiting spatial redundancy within one frame and temporal redundancy between consecutive frames. CBR uses a fixed encoding rate; VBR changes its rate with scene complexity.

The main challenge is continuous playout: the client must display content at the original timing even though network delay and available bandwidth vary. A client-side buffer absorbs jitter. If the buffer empties, playback stalls; if it overflows, capacity is wasted.



same content encoded at multiple rates

DASH selects encoded chunks and uses buffering to absorb rate changes

DASH and content distribution networks

2.12 DASH: Dynamic Adaptive Streaming over HTTP

In DASH, the server divides a video into chunks and stores each chunk at several encoding rates. A manifest file lists the URLs and available rates. The client periodically estimates available bandwidth and requests one chunk at a time at the highest sustainable quality.

- The client decides when to request the next chunk.
- The client selects a rate based on measured bandwidth and buffer level.
- The client can request chunks from different servers or CDN nodes.
- The client lowers the rate when bandwidth falls and raises it when conditions improve.
- DASH uses ordinary HTTP infrastructure, which works well with caches, proxies, and firewalls.

2.13 Content distribution networks

A single very large video server creates a single point of failure, a congestion point, and long paths to distant users. A CDN stores copies at geographically distributed nodes and directs users to a nearby or lightly loaded copy.

A CDN may use an enter-deep design with many servers inside access networks or a bring-home design with fewer large clusters near access networks. DNS, URL redirection, or an application manifest can direct a user to the selected CDN node.

A typical CDN access sequence is: the user receives a video URL, DNS maps the service name or alias to a CDN name, the CDN returns a nearby server address, and the client requests DASH chunks from that server.

OTT means over-the-top service: the application provider delivers content over the general Internet rather than operating the access ISP. It must choose what content to place at each node, which node should serve a user, and what rate the client can sustain.

UNIT 2 | SOCKET PROGRAMMING

2.14 UDP sockets

UDP provides an unreliable datagram service. There is no connection setup. The sender attaches the destination IP address and port to every datagram. The receiver obtains the sender address and port with the received data. Datagrams may be lost or arrive out of order.

- Client creates a UDP socket.
- Client sends a datagram to server IP and port.
- Server receives the datagram and learns the client address.
- Server processes the message and sends a reply to that address.
- Client receives the reply and closes the socket.

UDP pseudo-code

Client: create SOCK_DGRAM socket -> read input -> sendto(server address, port) -> recvfrom() -> display reply -> close. Server: create SOCK_DGRAM socket -> bind to a port -> loop over recvfrom() -> process data -> sendto(client address) .

2.15 TCP sockets

TCP provides a reliable, in-order byte stream. The client must contact a server that is already listening. The server has a welcoming socket and creates a new connected socket for each client, so one server can communicate with many clients.

- Server creates a TCP socket, binds it to a port, and calls listen().
- Server calls accept() and waits for a connection request.
- Client creates a TCP socket and calls connect(server IP, port).
- TCP establishes the connection. The server returns a new connection socket.
- Client and server exchange bytes using send and recv.
- The connected socket closes after the exchange; the welcoming socket can continue accepting clients.

UDP versus TCP socket interaction

UDP is message-oriented and every send includes a destination. TCP is connection-oriented and the connection identifies the peer after setup. TCP preserves order and reliability but adds setup and control overhead. UDP is simpler and faster to start, but the application must tolerate or handle loss, ordering, and timing itself.

Timeouts

A socket timeout prevents a program from waiting forever for a reply. The program sets a timeout, performs a blocking receive inside a try block, and handles the timeout exception in an except block. Timeouts are useful for retransmission, failure detection, and servers that should stop serving a silent client.

SOLVED PREVIOUS-YEAR QUESTIONS

The following answers are based on the supplied 2025 Computer Networks and 2025 Special Paper question sheets. In the exam, draw the small diagram shown in the relevant notes section and then write the numbered explanation.

PYQ 1 | Host, end system, and Web server

Answer. A host is a network-connected device that has an address and can send or receive data. An end system is a host at the network edge that runs application programs. Therefore, in the Internet context, host and end system are often used interchangeably, although host emphasises the device and end system emphasises its role as a source or destination of application data.

- Examples include laptops, smartphones, tablets, Web servers, mail servers, cloud servers, sensors, cameras, and smart appliances.
- A client process initiates a request, while a server process waits for requests and provides a service.
- A Web server is an end system when it is a connected host running Web-server software such as Apache or another HTTP server.
- Routers and switches are network-core devices. They forward packets but normally do not run the user application that creates the message.

Conclusion: a Web server is both a host/end system and a server process or service, depending on whether the question refers to hardware or software.

PYQ 2 | Circuit switching versus packet switching; TDM versus FDM

Answer. Circuit switching reserves resources for a complete call, so it provides predictable performance after setup and avoids queueing among calls using separate reserved resources. Packet switching shares resources on demand, so it is efficient for bursty traffic but may experience queueing, variable delay, and packet loss during congestion.

Circuit switching	Packet switching
Dedicated capacity and predictable service after setup.	Capacity shared dynamically among packets.
Can waste capacity when the user is silent.	Efficient for bursty and intermittent traffic.
Requires setup and teardown.	Usually no circuit setup; packets carry addresses.
Traditional telephone network.	Internet data network.

In FDM, each call gets a separate frequency band for the duration of the call. In TDM, time is divided into repeating slots and each call gets one or more slots. TDM can use the full link rate during its assigned slot, while FDM limits the call to the allocated band. TDM is often easier to allocate digitally and avoids frequency guard-band issues, but it requires synchronised time slots.

PYQ 3 | Three phases and delays in circuit/packet switching

Answer. Circuit switching has setup, data transfer, and teardown phases. Setup reserves resources along the path. Data transfer uses the established circuit. Teardown releases the resources.

Circuit switching has predictable delay and no packet loss caused by competing calls after reservation, but it has setup delay, can waste idle capacity, and is less efficient for bursty data. Packet switching has efficient sharing and no dedicated setup, but it can produce processing, queueing, transmission, and propagation delay. Queue overflow causes loss and retransmissions can increase delay.

PYQ 4 | End-to-end delay components

Answer. For a fixed route, the end-to-end delay is the sum of the delay components at every hop plus any end-system processing:

$$d_{\text{end-to-end}} = \text{sum}(d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}) \text{ over all links and routers}$$

Processing delay, transmission delay, and propagation delay are usually approximately constant for a fixed route and fixed packet/link conditions. Queueing delay is variable because it depends on traffic. Processing can also vary slightly with router load, but queueing is the main variable component.

PYQ 5 | Numerical delay: 1500 bytes over 10 Mbps

Given: $L = 1500$ bytes = 12000 bits, $R = 10,000,000$ bits/s, $d = 1000$ km = 1,000,000 m, $s = 2 \times 10^8$ m/s, queueing = 2 ms, processing = 1 ms.

$$\text{Transmission} = L/R = 12000/10,000,000 = 0.0012 \text{ s} = 1.2 \text{ ms}$$

$$\text{Propagation} = d/s = 1,000,000/(2 \times 10^8) = 0.005 \text{ s} = 5 \text{ ms}$$

$$\text{Total} = 1.2 + 5 + 2 + 1 = 9.2 \text{ ms}$$

PYQ 6 | Head-of-line blocking in HTTP/1.1 and HTTP/2

Answer. In HTTP/1.1 pipelining, the server sends responses in request order. If the first requested object is large or its data is delayed, smaller objects behind it wait even when they are ready. This is head-of-line blocking.

- HTTP/2 divides objects into frames.
- The server interleaves frames from different objects and can use priorities.
- Small objects can finish before a large object, reducing application-level waiting.
- HTTP/2 still runs over one TCP connection, so loss of a TCP segment can stall all streams until TCP recovers it.
- HTTP/3 uses QUIC over UDP to reduce cross-stream blocking and combine transport security with the connection.

PYQ 7 | Web caching: all objects or some objects?

Answer. A browser sends a request to a Web cache. If the object is cached and fresh, the cache returns it. Otherwise it retrieves the object from the origin server, stores a copy if allowed, and returns it to the browser. The cache reduces delay because it is close to the user and reduces traffic on the access link.

Caching does not reduce delay for every object. It helps only for cacheable objects that are already present and fresh, producing a cache hit. A cache miss still requires the origin-server path. Dynamic, private, no-store, or expired objects may not be served from the cache. Therefore, only the hit fraction benefits immediately. Conditional GET can validate a cached object without retransmitting its body.

PYQ 8 | P2P architecture and client/server terminology

Answer. P2P is a distributed architecture in which end systems communicate directly. A peer can request pieces from other peers and upload pieces to them. Unlike a pure client-server system, a peer is not permanently only a client or only a server.

- P2P provides self-scalability: a new peer adds demand but can also add upload capacity.
- Peers may be intermittent and may have changing IP addresses, making management and security difficult.
- A tracker or distributed discovery helps peers find one another; a central coordination service may still exist even though data transfer is peer-to-peer.
- In a communication session, the roles are logical and can change. The same peer may be a client while requesting one piece and a server while uploading another.

Conclusion: the statement that there is no fixed notion of client and server is correct for data exchange in a P2P swarm, although client and server processes can still exist at particular moments.

PYQ 9 | Why HTTP, SMTP, and IMAP use TCP

Answer. HTTP transfers Web objects, SMTP transfers e-mail messages between mail servers, and IMAP retrieves and synchronises messages. All require the application data to arrive without corruption, loss, duplication, or reordering. TCP provides checksum, sequence numbers, acknowledgements, retransmission, ordered byte-stream delivery, flow control, and congestion control. The cost is connection setup and control overhead.

PYQ 10 | Why voice and video may use TCP today

Answer. Real-time voice and video traditionally favour UDP because late data is often less useful than lost data, and UDP avoids retransmission delay. However, many modern services use TCP-based HTTPS streaming, adaptive video over HTTP, or QUIC over UDP. TCP works through NATs and firewalls, provides reliability, and fits Web/CDN infrastructure. DASH can request later chunks at a lower rate when the network is congested. For truly interactive real-time media, UDP or QUIC is often preferred so the application can control delay and loss.

PYQ 11 | SMTP e-mail journey through OSI and TCP/IP

Answer. At the sender, Alice writes the message in a user agent. SMTP transfers it to her mail server and later from her mail server to Bob mail server. Bob uses IMAP or Web mail to retrieve it.



Use this diagram for the e-mail path

OSI trace:

- Application: SMTP creates commands and e-mail content; DNS may locate the recipient mail server; IMAP later retrieves the message.
- Transport: TCP creates a reliable connection, segments the data, numbers bytes, acknowledges received data, and retransmits lost segments.
- Network: IP encapsulates TCP segments into datagrams and routes them between hosts.
- Data link: Ethernet or Wi-Fi frames carry each IP datagram across the next link.
- Physical: copper, fibre, or radio transmits the bits. At each router, the link-layer frame changes for the next hop; the IP datagram continues toward the destination.

TCP/IP trace:

SMTP or IMAP at application -> TCP at transport -> IP at network -> Ethernet/Wi-Fi at link -> physical bits. The receiver decapsulates in reverse order. The message may pass through a mail server, so there can be one SMTP TCP connection from user agent to sender server, another SMTP connection between mail servers, and an IMAP/HTTPS connection when Bob reads it.

PYQ 12 | Six-node full circuit connectivity

Given: six nodes A, B, C, D, E, F; one dedicated circuit for every pair; setup time 15 seconds per circuit.

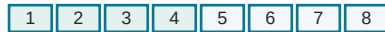
$$\text{Number of circuits} = n(n - 1)/2 = 6 \times 5 / 2 = 15 \text{ circuits}$$

If circuits are established one after another, total setup time = $15 \times 15 = 225$ seconds. If the switching system can establish all independent circuits in parallel, the elapsed setup time is 15 seconds. Since the question asks total setup time for all circuits, the usual sequential assumption gives 225 seconds; state the assumption.

The 128 Kbps requirement affects capacity reserved per circuit, not the number of pairwise circuits or the sequential setup-time calculation.

PYQ 13 | Go-Back-N and Selective Repeat

Answer. Both are pipelined reliable data-transfer protocols. The sender may transmit several packets before waiting, within a sender window. Sequence numbers identify packets, acknowledgements confirm delivery, and a timer detects loss.

Sliding-window reliability

GBN: error may retransmit the missing packet and all later packets

SR: receiver buffers correct packets and retransmits only the missing packet

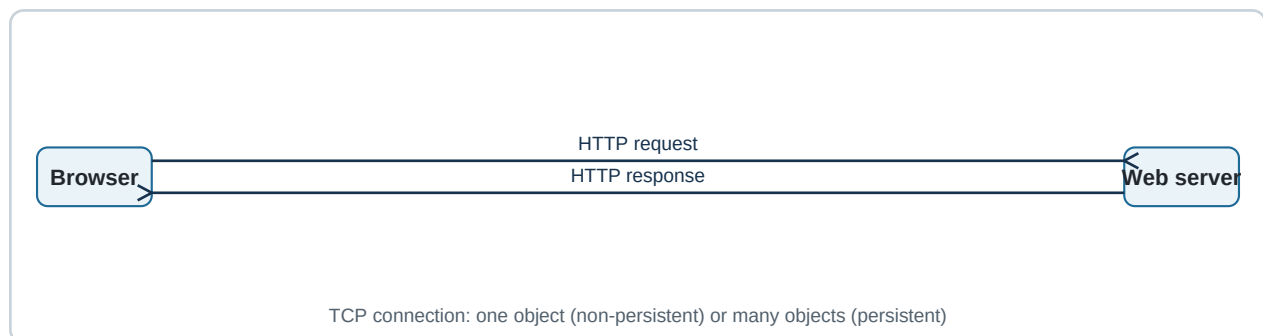
Use this diagram with a labelled sender and receiver window

Go-Back-N: the receiver normally accepts only the next in-order packet and sends cumulative ACKs. If packet k is lost, later packets may be discarded or not cumulatively accepted. On timeout, the sender retransmits k and all later unacknowledged packets in the window.

Selective Repeat: the receiver individually acknowledges correct packets and buffers out-of-order packets. When packet k is lost, the sender retransmits only k after its timer expires or a selective negative acknowledgement. SR uses bandwidth more efficiently but requires more receiver storage and more complex bookkeeping.

PYQ 14 | HTTP request-to-response for non-persistent HTTP

Answer. The user enters a URL. The browser obtains the server IP address using DNS. For the requested object, the browser opens a TCP connection to port 80. The server accepts the connection. The browser sends a GET request containing the URL path. The server returns a response with a status line, headers, and the object. The server closes the TCP connection. For every additional image or object, the browser repeats the process.



Each object gets its own TCP connection in non-persistent HTTP

For one object, response time is approximately $2RTT + \text{file transmission time}$: one RTT for TCP setup and one RTT for the HTTP request plus the first response bytes. Persistent HTTP reuses the connection and avoids repeated setup.

PYQ 15 | SMTP sequence and e-mail retrieval

SMTP sequence: TCP connection to port 25 -> server greeting 220 -> client HELO/EHLO -> MAIL FROM -> RCPT TO -> DATA -> headers and body terminated by a line containing a period -> server 250 acceptance -> QUIT -> server 221 closing response.

The receiving mail server stores the message in the recipient mailbox. The recipient uses IMAP to list, read, organise, and delete messages on the server, or uses Web mail where a browser communicates with a Web interface using HTTP/HTTPS.

PYQ 16 | Iterative and recursive DNS for www.example.com

Iterative answer. The client asks its local DNS. If the local server has no cache entry, it asks a root server. The root replies with the .com TLD server. The local server asks the TLD server, which replies with the authoritative server for example.com. The local server asks the authoritative server, receives the A record for www.example.com, caches it, and returns the IP to the client.

Recursive answer. The client asks its local DNS and requests recursion. The local DNS contacts the root, TLD, and authoritative servers on behalf of the client and returns the final address. Recursive service is

simpler for the client but places more work on the resolver.

Because the question says no caching, write all hierarchy contacts. If caching were present, the local resolver could answer from an unexpired record and skip some contacts.

PYQ 17 | Recommend an ISP tier for a small business

Answer. Recommend a Tier-2 or regional ISP for most small businesses. It usually provides reliable access, support, and adequate bandwidth while buying upstream connectivity from larger providers. A Tier-1 ISP is a global backbone and is expensive and unnecessary unless the business itself needs to provide transit or operate a very large network. A Tier-3 access provider may be sufficient for a small office but may offer less reach or resilience.

The final choice depends on bandwidth, service-level agreement, redundancy, public IP requirements, price, and geographic availability. The key reason is to match the scale of the provider to the scale of the business.

PYQ 18 | BitTorrent client with no data chunks

Answer. The new peer first contacts a tracker or peer-discovery system to obtain a list of peers in the swarm. It opens connections to several peers and learns which pieces each peer has through a bitfield or piece messages. Since it has no pieces, it requests pieces that are available from neighbours, preferably rarest-first to increase diversity. It verifies each downloaded piece using the torrent hash, advertises it, and uploads it to other peers. It uses optimistic unchoking to discover good upload partners. After receiving all pieces, it becomes a seed.

PYQ 19 | TCP versus UDP and applications

TCP: reliable, connection-oriented, in-order byte stream. It uses acknowledgements, retransmissions, flow control, congestion control, and connection setup. Applications: HTTP/HTTPS, SMTP, IMAP, FTP, and many Web transactions.

UDP: connectionless datagrams with low overhead. It does not provide reliability, ordering, flow control, congestion control, timing, or security. Applications: DNS queries, DHCP, real-time games, voice/video systems that choose UDP, and QUIC as a substrate for HTTP/3.

A protocol choice depends on the application. Reliability is useful for files and mail. Low delay and application-controlled loss handling may make UDP or QUIC useful for interactive media.

PYQ 20 | HTTP, SMTP, IMAP, voice, and video transport choice

HTTP, SMTP, and IMAP use TCP because these applications need complete, ordered data. Traditional interactive voice/video often uses UDP because retransmitting late packets can increase delay and cause poor interaction. Modern video-on-demand commonly uses TCP-based HTTP with buffering and adaptive bitrate, while HTTP/3 uses QUIC over UDP. Thus the correct answer recognises both the traditional real-time reason and the current Web/CDN deployment pattern.

FINAL REVISION SHEET

Definitions to memorise

- Protocol: rules for message format, order, and actions.
- Host/end system: a network-edge device that runs applications and sends or receives data.
- Forwarding: local movement of a packet from input link to output link.
- Routing: determination of source-to-destination paths.
- Transmission delay: time to push packet bits into the link.
- Propagation delay: time for the signal to travel across the medium.
- Throughput: delivered bit rate; the bottleneck limits the path.
- Socket: application-to-transport interface.
- HTTP: Web request-response protocol.
- SMTP: server-to-server e-mail transfer protocol.

- IMAP: server-based e-mail retrieval and synchronisation protocol.
- DNS: distributed hierarchical name-to-address database and protocol.
- DASH: adaptive HTTP streaming using encoded chunks and a manifest.
- CDN: distributed infrastructure that stores content near users.

Diagrams worth drawing

- Internet edge-core diagram with hosts, access networks, and routers.
- Packet switching versus circuit switching comparison.
- Four delay components and their formulas.
- OSI and TCP/IP layer mapping.
- Encapsulation: message -> segment -> datagram -> frame.
- Client-server and P2P architecture.
- HTTP request and response.
- SMTP mail-server path and IMAP retrieval.
- DNS root -> TLD -> authoritative hierarchy.
- DASH client buffer and CDN chunk delivery.
- Go-Back-N versus Selective Repeat windows.

Final exam strategy

For a 10-mark answer, spend about one paragraph on the definition, one labelled diagram, three to five numbered working steps, and a short comparison or conclusion. In numerical problems, write the formula, substitute with units, and box the final answer.